

LAPORAN PRATIKUM PEKAN 9

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR	i
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum	2
BAB II	3
PEMBAHASAN	3
2.1 Kode Program	3
2.2 Langkah Kerja	11
2.3 Analisis Hasil	19
BAB III.....	19
PENUTUP.....	19
3.1 Kesimpulan	19
3.2 Saran	19
DAFTAR PUSTAKA.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Program

a. Kode program BTree_2511531004

```
2  package Pekan9_2511531004;
3
4  public class BTree_2511531004 {
5      private Node_2511531004 root_1004;
6      private Node_2511531004 currentNode;
7
8      public BTree_2511531004 () {
9          root_1004 = null;
10     }
11
12     public boolean search (int data_1004) {
13         return search (root_1004, data_1004);
14     }
15
16     private boolean search (Node_2511531004 node, int data_1004) {
17         if (node.getData_2511531004() == data_1004)
18             return true;
19         if (node.getLeft_2511531004() != null )
20             if (search (node.getLeft_2511531004() , data_1004))
21                 return true;
22         if (node.getRight_2511531004() != null )
23             if (search (node.getRight_2511531004() , data_1004))
24                 return true;
25         return false;
26     }
27
28     public void printInorder_2511531004() {
29         root_1004.printInorder_2511531004(root_1004);
30     }
31     public void printPreorder_2511531004() {
32         root_1004.printPreorder_2511531004(root_1004) ;
33     }
34     public void printPostorder_2511531004() {
35         root_1004.printPostorder_2511531004(root_1004);
36     }
37     public Node_2511531004 getRoot_2511531004() {
38         return root_1004;
39     }
40
41     public boolean isEmpty_2511531004 () {
42         return root_1004 == null;
43     }
44
45     public int countNodes_2511531004 () {
46         return countNodes_2511531004 (root_1004);
47     }
48 }
```

```

49     private int countNodes_2511531004 (Node_2511531004 node_1004) {
50         int count = 1;
51         if (node_1004 == null) {
52             return 0;
53         }
54         else {
55             count += countNodes_2511531004
56 (node_1004.getLeft_2511531004());
57             count += countNodes_2511531004
58 (node_1004.getRight_2511531004());
59             return count;
60         }
61     }
62
63     public void print_2511531004 () {
64         root_1004.print_2511531004 () ;
65     }
66
67     public Node_2511531004 getCurrent_2511531004() {
68         return currentNode;
69     }
70
71     public void setCurrent_2511531004 (Node_2511531004 node_1004) {
72         this.currentNode = node_1004;
73     }
74
75     public void setRoot_2511531004 (Node_2511531004 root_1004) {
76         this.root_1004 = root_1004;
77     }

```

b. Kode Program Btree_Driver_2511531004

```

package Pekan9_2511531004;

public class BTreeDriver_2511531004 {
    public static void main(String [] args) {
        // membuat pohon

        BTree_2511531004 tree_1004 = new BTree_2511531004() ;
        System.out.print("Jumlah simpul awal pohon : ");
        System.out.println(tree_1004.countNodes_2511531004());

        Node_2511531004 root_1004 = new Node_2511531004 (1);
        // menambahkan simpul data
        tree_1004.setRoot_2511531004(root_1004);
        System.out.print("Jumlah simpul jika hanya ada root : ");
        System.out.println(tree_1004.countNodes_2511531004());
        Node_2511531004 node2 = new Node_2511531004 (2);
        Node_2511531004 node3 = new Node_2511531004 (3);
        Node_2511531004 node4 = new Node_2511531004 (4);
        Node_2511531004 node5 = new Node_2511531004 (5);
        Node_2511531004 node6 = new Node_2511531004 (6);
        Node_2511531004 node7 = new Node_2511531004 (7);
    }
}

```

```

        Node_2511531004 node8 = new Node_2511531004 (8);
        Node_2511531004 node9 = new Node_2511531004 (9);

        root_1004.setLeft_2511531004(node2);
        node2.setRight_2511531004(node4);
        node2.setRight_2511531004 (node5);
        node4.setRight_2511531004(node8);
        root_1004.setRight_2511531004(node3);
        node3.setLeft_2511531004(node6);
        node3.setRight_2511531004(node7);
        node6.setLeft_2511531004 (node9);

        //set root

        tree_1004.setCurrent_2511531004(tree_1004.getRoot_2511531004());
        System.out.print("Menampilkan simpul terakhir : ");

        System.out.println(tree_1004.getCurrent_2511531004().getData_2511
531004());
        System.out.print ("jumlah simpul ; setelah simpul 7
ditambahkan : ");
        System.out.println(tree_1004.countNodes_2511531004());
        System.out.print("InOrder : ");
        tree_1004.printInorder_2511531004();
        System.out.print("\nPostOrder : ");
        tree_1004.printPostorder_2511531004();
        System.out.println("\nMenampilkan simpul dalam bentuk
pohon");
        tree_1004.print_2511531004();

    }
}

```

c. Kode Program GraphTraversal_2511531004

```

package Pekan9_2511531004;
import java.util.*;

public class GraphTraversal_2511531004 {
    private Map<String, List<String>> graph = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_2511531004(String node1, String node2) {
        graph.putIfAbsent(node1, new ArrayList<>());
        graph.putIfAbsent(node2, new ArrayList<>());
        graph.get(node1).add(node2);
    }
}

```



```

        graph.get(node2).add(node1);
    }

    // Menampilkan graf awal
    public void printGraph_2511531004() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node : graph.keySet()) {
            System.out.print(node + " -> ");
            List<String> neighbors = graph.get(node);
            System.out.println(String.join(", ", neighbors));
        }
        System.out.println();
    }

    // DFS rekursif
    public void dfs_2511531004(String start) {
        Set<String> visited = new HashSet<>();
        System.out.println("Penelusuran DFS:");
        dfsHelper_2511531004(start, visited);
        System.out.println();
    }

    private void dfsHelper_2511531004(String current, Set<String>
visited) {
        if (visited.contains(current)) return;
        visited.add(current);
        System.out.print(current + " ");
        for (String neighbor : graph.getDefault(current, new
ArrayList<>())) {
            dfsHelper_2511531004(neighbor, visited);
        }
    }

    // BFS iteratif
    public void bfs_2511531004(String start) {
        Set<String> visited_1004 = new HashSet<>();
        Queue<String> queue_1004 = new LinkedList<>();
        queue_1004.add(start);
        visited_1004.add(start);
        System.out.println("Penelusuran BFS:");
        while (!queue_1004.isEmpty()) {
            String current = queue_1004.poll();
            System.out.print(current + " ");
            for (String neighbor : graph.getDefault(current, new
ArrayList<>())) {
                if (!visited_1004.contains(neighbor)) {
                    queue_1004.add(neighbor);
                    visited_1004.add(neighbor);
                }
            }
        }
        System.out.println();
    }

    // Main
    public static void main(String[] args) {

```

```

        GraphTraversal_2511531004 graph = new
GraphTraversal_2511531004();

        // Contoh graf: A-B, A-C, B-D, B-E
graph.addEdge_2511531004("A", "B");
graph.addEdge_2511531004("A", "C");
graph.addEdge_2511531004("B", "D");
graph.addEdge_2511531004("B", "E");

        // Cetak graf awal
System.out.println("Graf Awal adalah: ");
graph.printGraph_2511531004();

        // Lakukan penelusuran
graph.dfs_2511531004("A");
graph.bfs_2511531004("A");
    }
}

```

d. Kode Program Node_2511531004

```

package Pekan9_2511531004;

public class Node_2511531004 {
    int data_1004;
    Node_2511531004 left_1004;
    Node_2511531004 right_1004;

    public Node_2511531004 (int data_1004) {
        this.data_1004 = data_1004;
        left_1004 = null;
        right_1004 = null;
    }

    public void setLeft_2511531004 (Node_2511531004 node_1004) {
        if (left_1004 == null)
            left_1004 = node_1004;
    }

    public void setRight_2511531004 (Node_2511531004 node_1004) {
        if (right_1004 == null)
            right_1004 = node_1004;
    }

    public Node_2511531004 getLeft_2511531004 () {
        return left_1004;
    }

    public Node_2511531004 getRight_2511531004 () {
        return right_1004;
    }

    public int getData_2511531004 () {
        return data_1004;
    }

    public void setData_2511531004 (int data_1004) {
        this.data_1004 = data_1004;
    }
}

```

```

void printPreorder_2511531004 (Node_2511531004 node_1004) {
    if (node_1004 == null)
        return;
    System.out.print(node_1004.data_1004 + " ");
    printPreorder_2511531004 (node_1004.left_1004);
    printPreorder_2511531004 (node_1004.right_1004);
}

void printPostorder_2511531004 (Node_2511531004 node_1004) {
    if (node_1004 == null)
        return;
    printPostorder_2511531004 (node_1004.left_1004);
    printPostorder_2511531004 (node_1004.right_1004);
    System.out.print(node_1004.data_1004 + " ");
}

void printInorder_2511531004 (Node_2511531004 node_1004) {
    if (node_1004 == null)
        return;
    printInorder_2511531004 (node_1004.left_1004);
    System.out.print(node_1004.data_1004 + " ");
    printInorder_2511531004 (node_1004.right_1004);
}

public String print_2511531004() {
    return this.print_2511531004("", true, "");
}

public String print_2511531004(String prefix_1004, boolean
isTail_1004, String sb_1004) {
    if (right_1004 != null) {
        right_1004.print_2511531004(prefix_1004 + (isTail_1004 ? "|"
" : "  "), false, sb_1004);
    }
    System.out.println(prefix_1004 + (isTail_1004 ? "\\--" : "/--")
+ data_1004);
    if (left_1004 != null) {
        left_1004.print_2511531004(prefix_1004 + (isTail_1004 ? "
" : "|  "), true, sb_1004);
    }
    return sb_1004;
}
}

```

2.2 Pembahasan Kode Program

a. BTree_2511531004

Deklarasi & Constructor

- Ada root_1004 (akar tree) dan currentNode.
- Constructor set root_1004 = null → tree kosong saat pertama kali dibuat.

Search Rekursif

- Method search(int data) mulai dari root.
- Rekursif cek:
 - Kalau data sama → return true.
 - Kalau tidak, cek anak kiri → kalau ketemu return true.
 - Kalau tidak, cek anak kanan → kalau ketemu return true.
 - Kalau semua gagal → return false.
- Jadi pencarian dilakukan **DFS rekursif** ke seluruh node.

Traversal

- Ada tiga traversal standar:
 - **Inorder** → kiri, root, kanan.
 - **Preorder** → root, kiri, kanan.
 - **Postorder** → kiri, kanan, root.
- Semua dipanggil lewat method di class Node_2511531004.

Count Nodes

- Method countNodes_2511531004() hitung jumlah node dengan rekursi.
- Kalau node null → return 0.
- Kalau ada node → hitung dirinya + anak kiri + anak kanan.

- Jadi hasilnya jumlah total node dalam tree.

Print

- Method `print_2511531004()` panggil fungsi print dari node.
- Biasanya dipakai untuk menampilkan struktur tree ke console.

Getter & Setter

- `getRoot_2511531004()` → ambil root.
- `isEmpty_2511531004()` → cek apakah tree kosong.
- `getCurrent_2511531004()` / `setCurrent_2511531004()` → atur node aktif.
- `setRoot_2511531004()` → set akar tree dari luar class.

b. BTreeDriver_2511531004

Buat objek tree kosong

java

```
BTree_2511531004 tree_1004 = new BTree_2511531004();
System.out.println(tree_1004.countNodes_2511531004());
```

→ Tree baru, root masih null, jumlah simpul = 0.

Tambah root

java

```
Node_2511531004 root_1004 = new Node_2511531004(1);
tree_1004.setRoot_2511531004(root_1004);
```

→ Root diisi dengan data 1. Jumlah simpul = 1.

Tambah node lain

java

```
Node_2511531004 node2 = new Node_2511531004(2);
```

```
Node_2511531004 node3 = new Node_2511531004(3);
```

```
Node_2511531004 node4 = new Node_2511531004(4);
```

```
Node_2511531004 node5 = new Node_2511531004(5);
```

```
Node_2511531004 node6 = new Node_2511531004(6);
```

```
Node_2511531004 node7 = new Node_2511531004(7);
```

```
Node_2511531004 node8 = new Node_2511531004(8);
```

```
Node_2511531004 node9 = new Node_2511531004(9);
```

→ Bikin node dengan data 2–9.

Hubungkan node jadi pohon

→ Root punya anak kiri (2) dan kanan (3). → Node 2 punya anak kanan (akhirnya jadi node5, karena baris sebelumnya ke-overwrite). → Node 3 punya anak kiri (6) dan kanan (7). → Node 6 punya anak kiri (9). → Node 4 sempat dikasih anak kanan (8), tapi karena node4 nggak pernah dipasang ke tree (di-overwrite oleh node5), node8 nggak kepakai.

Set current node

```
tree_1004.setCurrent_2511531004(tree_1004.getRoot_2511531004());
```

```
System.out.println(tree_1004.getCurrent_2511531004().getData_2511531004());
```

→ Current node = root (data 1).

Hitung jumlah simpul

```
java
```

```
System.out.println(tree_1004.countNodes_2511531004());
```

→ Menghitung semua node yang terhubung dari root.

Traversal

java

```
tree_1004.printInorder_2511531004();
```

```
tree_1004.printPostorder_2511531004();
```

→ Menampilkan node sesuai urutan traversal.

Print tree

java

```
tree_1004.print_2511531004();
```

→ Menampilkan struktur tree dalam bentuk teks.

c. GraphTraversal_2511531004

Struktur Data

java

```
private Map<String, List<String>> graph = new HashMap<>();
```

- Graf disimpan dalam bentuk **Adjacency List**: setiap node punya daftar tetangga.

Tambah Edge

java

```
public void addEdge_2511531004(String node1, String node2) {
```

```
    graph.putIfAbsent(node1, new ArrayList<>());
```

```
    graph.putIfAbsent(node2, new ArrayList<>());
```

```
    graph.get(node1).add(node2);
```

```
    graph.get(node2).add(node1);
```

```
}
```

- Menambahkan sisi (edge) antara dua node.
- Karena graf tak berarah, node1 → node2 dan node2 → node1.

Cetak Graf

java

```
public void printGraph_2511531004() {  
    for (String node : graph.keySet()) {  
        System.out.print(node + " -> ");  
        System.out.println(String.join(", ", graph.get(node)));  
    }  
}
```

- Menampilkan adjacency list ke console.

DFS (Depth First Search)

java

```
public void dfs_2511531004(String start) {  
    Set<String> visited = new HashSet<>();  
    dfsHelper_2511531004(start, visited);  
}
```

- DFS rekursif: mulai dari node start, telusuri semua tetangga secara mendalam.
- dfsHelper akan berhenti kalau node sudah dikunjungi.

BFS (Breadth First Search)

java

```
public void bfs_2511531004(String start) {  
    Set<String> visited = new HashSet<>();  
    Queue<String> queue = new LinkedList<>();  
    queue.add(start);
```



```

visited.add(start);

while (!queue.isEmpty()) {

    String current = queue.poll();

    System.out.print(current + " ");

    for (String neighbor : graph.getDefault(current, new ArrayList<>())) {

        if (!visited.contains(neighbor)) {

            queue.add(neighbor);

            visited.add(neighbor);

        }

    }

}
}

```

- BFS iteratif: pakai **queue** untuk menelusuri node level demi level.

Main Method

```

java

public static void main(String[] args) {

    GraphTraversal_2511531004 graph = new GraphTraversal_2511531004();

    graph.addEdge_2511531004("A", "B");

    graph.addEdge_2511531004("A", "C");

    graph.addEdge_2511531004("B", "D");

    graph.addEdge_2511531004("B", "E");

    graph.printGraph_2511531004();

    graph.dfs_2511531004("A");
}

```

```
graph.bfs_2511531004("A");
}
```

- Bikin graf sederhana: A terhubung ke B dan C, B terhubung ke D dan E.
- Cetak adjacency list.
- Jalankan DFS dan BFS mulai dari node A.

d. Node_2511531004

Atribut

```
java
```

```
int data_1004;
```

```
Node_2511531004 left_1004;
```

```
Node_2511531004 right_1004;
```

- data_1004 → nilai yang disimpan di node.
- left_1004 → anak kiri.
- right_1004 → anak kanan.

Constructor

```
java
```

```
public Node_2511531004(int data_1004) {
```

```
    this.data_1004 = data_1004;
```

```
    left_1004 = null;
```

```
    right_1004 = null;
```

```
}
```

- Saat bikin node baru, data diisi, anak kiri/kanan kosong.

Setter & Getter

- `setLeft_2511531004()` dan `setRight_2511531004()` → pasang anak kiri/kanan kalau masih kosong.
- `getLeft_2511531004()`, `getRight_2511531004()`, `getData_2511531004()` → ambil data atau anak.
- `setData_2511531004()` → ubah nilai data.

Traversal

- `printPreorder_2511531004()` → root → kiri → kanan.
- `printPostorder_2511531004()` → kiri → kanan → root.
- `printInorder_2511531004()` → kiri → root → kanan. Semua traversal dilakukan rekursif.

Print Tree

java

```
public String print_2511531004() {
    return this.print_2511531004("", true, "");
}
```

- Method ini bikin tampilan tree dalam bentuk teks.
- Node kanan digambar dulu, lalu node root, lalu node kiri.
- Outputnya mirip diagram pohon dengan garis `|`, `\--`, `/--`.

DAFTAR PUSTAKA

1. Oracle. (2024). *Java™ Platform, Standard Edition Documentation*. Oracle.
<https://docs.oracle.com/javase/8/docs/api/> ([docs.oracle.com in Bing](#))
2. Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.
3. Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (11th ed.). Pearson.
4. Wirth, N. (2006). *Algorithms and Data Structures*. Springer. (Referensi umum untuk konsep queue, stack, dan linked list).
5. GeeksforGeeks. (2025). *Queue in Java – Implementation and Examples*.
<https://www.geeksforgeeks.org/queue-in-java/> ([geeksforgeeks.org in Bing](#))
6. Tutorialspoint. (2025). *Java – Collections Framework*.
https://www.tutorialspoint.com/java/java_collections.htm
([tutorialspoint.com in Bing](#))
7. Baeldung. (2025). *Guide to the Java Queue Interface*.
<https://www.baeldung.com/java-queue>

Tugas Struktur Data Pekan 9

a. Kode Program Tugas9_2511531004

```
package Pekan9_2511531004;

import javax.swing.*;
import java.awt.*;
import java.util.*;
import java.util.List;
import java.util.Map;
import java.util.HashMap;
import java.util.Arrays;
import java.util.Queue;
import java.util.LinkedList;
import java.util.Set;
import java.util.HashSet;
import java.util.Stack;

public class Tugas9_2511531004 extends JFrame {
    private Map<String, List<String>> graph_1004 = new HashMap<>();
    private JTextArea hasilArea_1004;
    private JComboBox<String> startBox_1004, goalBox_1004;
    private GraphPanel_1004 graphPanel_1004;
    private Set<String> visitedNodes_1004 = new HashSet<>();

    public Tugas9_2511531004() {
        setTitle("Pencarian Jalur BFS & DFS - 2511531004");
        setSize(800, 600);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        // Graph minimal 10 node, 15 edge
        graph_1004.put("Rektorat", Arrays.asList("Perpus", "FTI"));
        graph_1004.put("Perpus", Arrays.asList("Rektorat", "PKM",
"Masjid"));
        graph_1004.put("FTI", Arrays.asList("Rektorat", "Lab Jaringan",
"Gedung A"));
        graph_1004.put("PKM", Arrays.asList("Perpus", "Masjid", "Gedung
B"));
        graph_1004.put("Lab Jaringan", Arrays.asList("FTI", "Labor AI",
"Gedung C"));
        graph_1004.put("Masjid", Arrays.asList("Perpus", "PKM", "Labor
AI"));
        graph_1004.put("Labor AI", Arrays.asList("Lab Jaringan",
"Masjid", "Gedung D"));
        graph_1004.put("Gedung A", Arrays.asList("FTI", "Gedung B"));
        graph_1004.put("Gedung B", Arrays.asList("PKM", "Gedung A",
"Gedung C"));
        graph_1004.put("Gedung C", Arrays.asList("Lab Jaringan", "Gedung
B", "Gedung D"));
        graph_1004.put("Gedung D", Arrays.asList("Labor AI", "Gedung
C"));

        JPanel panel = new JPanel(new BorderLayout());
```

```

        // ComboBox Start & Goal
        JPanel topPanel = new JPanel();
        startBox_1004 = new JComboBox<>(graph_1004.keySet().toArray(new
String[0]));
        goalBox_1004 = new JComboBox<>(graph_1004.keySet().toArray(new
String[0]));
        topPanel.add(new JLabel("Start:"));
        topPanel.add(startBox_1004);
        topPanel.add(new JLabel("Goal:"));
        topPanel.add(goalBox_1004);

        // Tombol BFS, DFS, Reset
        JButton bfsBtn = new JButton("BFS");
        JButton dfsBtn = new JButton("DFS");
        JButton resetBtn = new JButton("Reset");

        bfsBtn.addActionListener(e -> jalankanBFS());
        dfsBtn.addActionListener(e -> jalankanDFS());
        resetBtn.addActionListener(e -> {
            hasilArea_1004.setText("");
            visitedNodes_1004.clear();
            graphPanel_1004.repaint();
        });

        topPanel.add(bfsBtn);
        topPanel.add(dfsBtn);
        topPanel.add(resetBtn);

        panel.add(topPanel, BorderLayout.NORTH);

        // Area hasil
        hasilArea_1004 = new JTextArea(5, 40);
        panel.add(new JScrollPane(hasilArea_1004), BorderLayout.SOUTH);

        // Panel visualisasi graph
        graphPanel_1004 = new GraphPanel_1004(graph_1004,
visitedNodes_1004);
        panel.add(graphPanel_1004, BorderLayout.CENTER);

        add(panel);
    }

    private void jalankanBFS() {
        String start = (String) startBox_1004.getSelectedItem();
        String goal = (String) goalBox_1004.getSelectedItem();
        List<String> path = bfs(start, goal);
        hasilArea_1004.setText("BFS Path: " + path + "\nJumlah Node: " +
path.size());
        visitedNodes_1004.addAll(path);
        graphPanel_1004.repaint();
    }

    private void jalankanDFS() {
        String start = (String) startBox_1004.getSelectedItem();
        String goal = (String) goalBox_1004.getSelectedItem();
        List<String> path = dfs(start, goal);
    }

```

```

        hasilArea_1004.setText("DFS Path: " + path + "\nJumlah Node: " +
path.size());
        visitedNodes_1004.addAll(path);
        graphPanel_1004.repaint();
    }

    // BFS
    private List<String> bfs(String start, String goal) {
        Queue<List<String>> queue = new LinkedList<>();
        queue.add(Arrays.asList(start));
        Set<String> visited = new HashSet<>();

        while (!queue.isEmpty()) {
            List<String> path = queue.poll();
            String node = path.get(path.size() - 1);
            if (node.equals(goal)) return path;
            if (!visited.contains(node)) {
                visited.add(node);
                for (String neighbor : graph_1004.getOrDefault(node, new
ArrayList<>())) {
                    List<String> newPath = new ArrayList<>(path);
                    newPath.add(neighbor);
                    queue.add(newPath);
                }
            }
        }
        return new ArrayList<>();
    }

    // DFS
    private List<String> dfs(String start, String goal) {
        Stack<List<String>> stack = new Stack<>();
        stack.push(Arrays.asList(start));
        Set<String> visited = new HashSet<>();

        while (!stack.isEmpty()) {
            List<String> path = stack.pop();
            String node = path.get(path.size() - 1);
            if (node.equals(goal)) return path;
            if (!visited.contains(node)) {
                visited.add(node);
                for (String neighbor : graph_1004.getOrDefault(node, new
ArrayList<>())) {
                    List<String> newPath = new ArrayList<>(path);
                    newPath.add(neighbor);
                    stack.push(newPath);
                }
            }
        }
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
Tugas9_2511531004().setVisible(true));
    }
}

```

```

// Panel untuk menggambar graph
class GraphPanel_1004 extends JPanel {
    private Map<String, List<String>> graph;
    private Set<String> visited;
    private Map<String, Point> posisiNode = new HashMap<>();

    public GraphPanel_1004(Map<String, List<String>> graph, Set<String>
visited) {
        this.graph = graph;
        this.visited = visited;

        // Posisi node manual
        posisiNode.put("Rektorat", new Point(100, 50));
        posisiNode.put("Perpus", new Point(50, 150));
        posisiNode.put("FTI", new Point(200, 150));
        posisiNode.put("PKM", new Point(100, 250));
        posisiNode.put("Lab Jaringan", new Point(250, 250));
        posisiNode.put("Masjid", new Point(50, 350));
        posisiNode.put("Labor AI", new Point(250, 350));
        posisiNode.put("Gedung A", new Point(350, 150));
        posisiNode.put("Gedung B", new Point(350, 250));
        posisiNode.put("Gedung C", new Point(350, 350));
        posisiNode.put("Gedung D", new Point(450, 350));
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

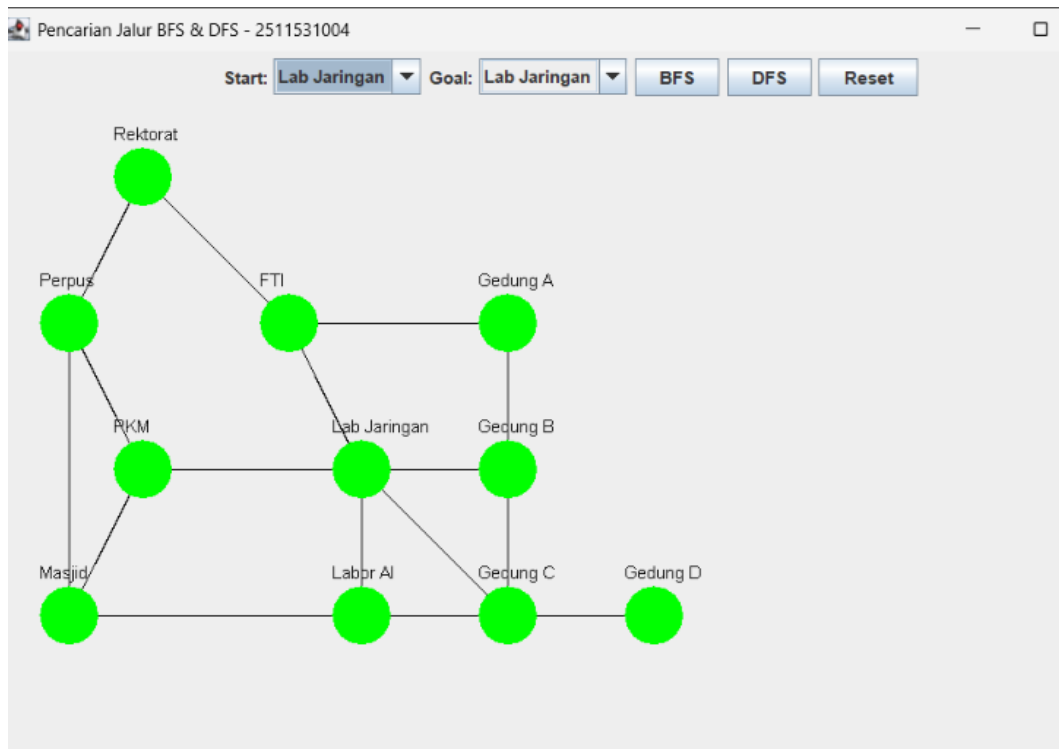
        // Gambar edge
        g.setColor(Color.BLACK);
        for (String node : graph.keySet()) {
            Point p1 = posisiNode.get(node);
            for (String neighbor : graph.get(node)) {
                Point p2 = posisiNode.get(neighbor);
                if (p1 != null && p2 != null) {
                    g.drawLine(p1.x, p1.y, p2.x, p2.y);
                }
            }
        }

        // Gambar node
        for (String node : graph.keySet()) {
            Point p = posisiNode.get(node);
            if (p != null) {
                if (visited.contains(node)) {
                    g.setColor(Color.RED); // node dikunjungi
                } else {
                    g.setColor(Color.GREEN); // node default
                }
                g.fillOval(p.x - 20, p.y - 20, 40, 40);
                g.setColor(Color.BLACK);
                g.drawString(node, p.x - 20, p.y - 25);
            }
        }
    }
}

```


}

Hasil Output:



Kesimpulan

Perbandingan BFS dan DFS

- **Cara kerja:**
 - **BFS** menelusuri graf **per level**. Mulai dari node awal, lalu semua tetangga terdekat, baru lanjut ke level berikutnya.
 - **DFS** menelusuri graf **sedalam mungkin**. Dari node awal, langsung masuk ke jalur terdalam sebelum balik lagi ke cabang lain.
- **Struktur data yang dipakai:**
 - **BFS** → pakai **Queue** (FIFO).
 - **DFS** → pakai **Stack** (LIFO) atau rekursi.
- **Jalur yang ditemukan:**

- **BFS** selalu menemukan **jalur terpendek** (dalam jumlah edge) dari start ke goal.
- **DFS** bisa menemukan jalur, tapi bukan jaminan jalur terpendek.
- **Kelebihan:**
 - **BFS** cocok untuk mencari shortest path dan analisis level node.
 - **DFS** lebih hemat memori di graf besar, dan berguna untuk eksplorasi penuh (misalnya topological sort, deteksi siklus).
- **Kekurangan:**
 - **BFS** bisa boros memori kalau graf sangat besar (karena queue menyimpan banyak jalur).
 - **DFS** bisa nyasar ke jalur panjang dulu, jadi kurang efisien kalau tujuannya dekat.